Algorithmic Design and Programming Fundamentals in Financial Applications

The objective of this assignment is to design a structured algorithm for a finance application that calculates a monthly budget. According to Simplilearn (2021), an algorithm is a clear and finite set of steps used to solve a specific problem. To create an effective tool, the programmer must ensure that the code is not only functional but also understandable for humans (MV, 2019). This essay will demonstrate how sequencing, selection, and iteration work together to manage a user's financial data accurately.

Algorithm Design: Sequencing and Input

To develop a functional finance application, the algorithm must follow a strict logical order known as sequencing. The process begins by prompting the user to enter their total monthly income, followed by their fixed expenses, such as rent and utilities. According to Simplilearn (2021), we must first provide some input and then follow a sequence of steps to acquire the required outcome. This order is essential because the computer performs tasks sequentially. As Computer Science Tutor (2022) states, steps have to be in a specific order to make logical sense and execute changes to data in an appropriate manner. For example, the program cannot display the final budget before it has successfully collected and processed the input data. Therefore, establishing a clear sequence from the "begin" keyword ensures that the mathematical computation is accurate.

Advanced Logic: Selection and Iteration

Once the initial data is collected, the algorithm employs advanced control structures to process complex information. To manage multiple variable expenses such as groceries or entertainment, the program uses **iteration**, which allows a set of instructions to repeat until a specific condition is met. This ensures the user can input as many items as necessary without restarting the application. Furthermore, the algorithm utilizes **selection** to make logical decisions based on the calculated data (Computer Science Tutor, 2022). According to Computer Science Tutor (2022), selection is used to make a choice in response to a condition, such as determining if the total expenses exceed the monthly income. To maximize computational efficiency, this selection process can be optimized using Boolean algebra. According to Ndjountche (2016), logic functions combine signals based on Boolean laws to simplify the execution flow . For example, by using a single combined expression such as checking if the budget is negative AND a flag for emergency expenses is absent the processor performs fewer operations. This operational approach is essential because, as noted in insight into theoretical and Applied Informatics, an effective algorithm must use the smallest number of memory cells and achieve minimum execution time . This logical manipulation is essential to achieve the desired result of providing an accurate financial summary (Chaudhuri, 2020).

Debugging the Logical Error

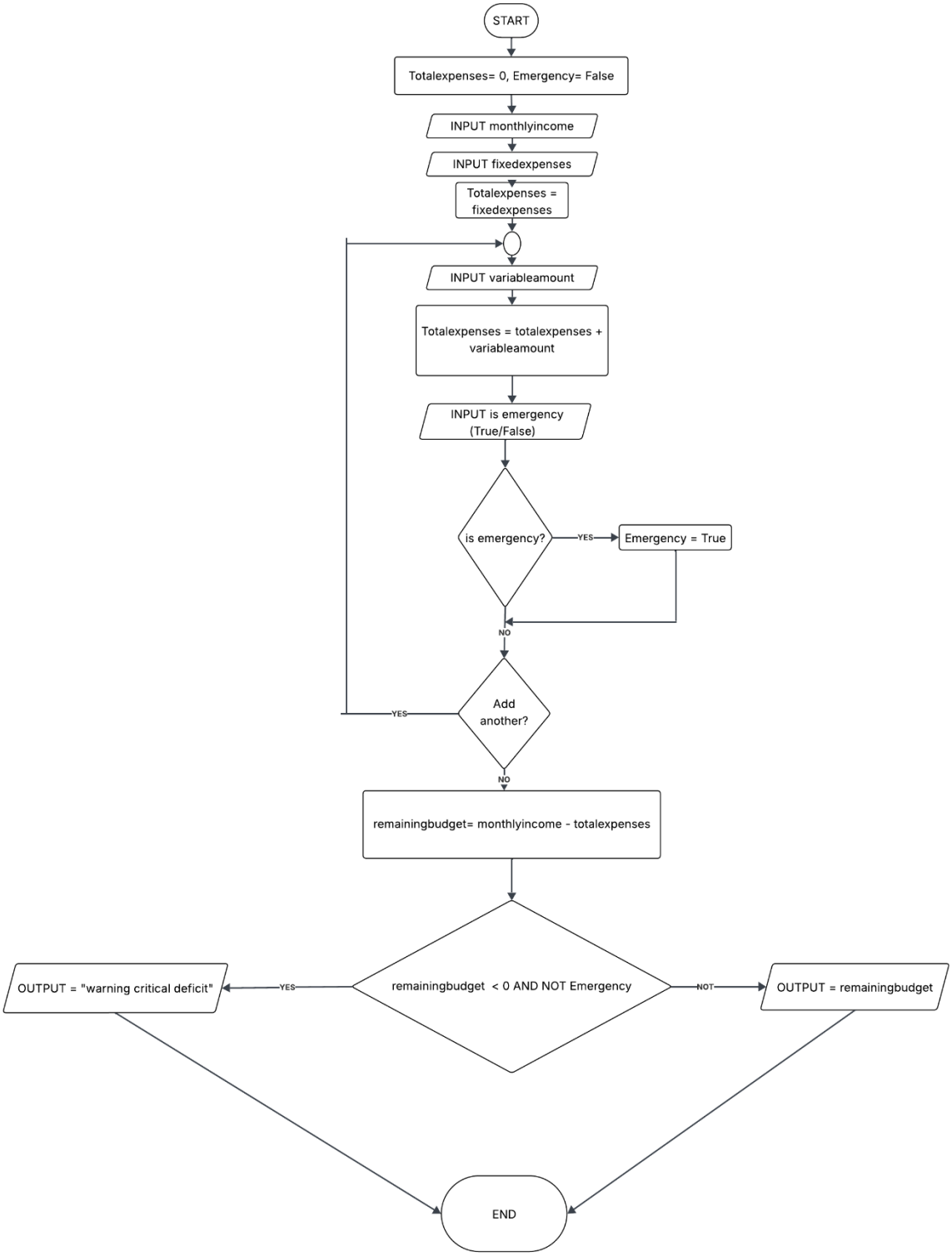
Even with a well-designed structure, a logical error may occur where the algorithm calculates an incorrect remaining budget. A logical error is particularly challenging because the program runs without crashing, but the output is not what the user expects (Computer Science Tutor, 2020). For instance, in this finance application, a common error could involve the algorithm failing to subtract a specific variable expense from the total income, leading to an inflated balance. To resolve this, I would implement several debugging techniques. According to Computer Science Tutor (2020), one of the most effective methods is "step-by-step execution," which involves running the code line by line to see exactly where the calculation goes wrong. Additionally, I would use "watch variables" to monitor how the budget value changes during each iteration of the loop. If the automated tools are not enough, I would perform "hand tracing" by manually calculating the values on paper to compare them with the computer's results. These systematic steps ensure that the logic is rectified and the final budget is accurate.

Conclusion

In conclusion, the development of a reliable finance application depends on the disciplined application of structured programming principles. By organizing the code into a logical sequence, a programmer ensures that the computer processes data in an order that makes sense. Furthermore, integrating advanced logic like iteration and selection allows the software to handle real world complexities, such as multiple expenses and budget limits, with

precision. Finally, using robust debugging techniques to identify logical errors ensures that the final product is not only functional but also accurate and trustworthy.

Figure 1. Optimized Monthly Budget Algorithm using Boolean Logic and Iterative Processes.



Note 1. Created by the author using Lucidchart (2024). *This flowchart illustrates a structured algorithm designed to calculate a user's monthly budget. It begins by initializing core variables and collecting primary income data. The process incorporates an iterative loop (repetition) to aggregate multiple variable expenses. Finally, it employs a Boolean logic gate to evaluate if a critical deficit exists—triggering a warning only if the budget is negative and no emergency was reported—ensuring computational efficiency and accurate financial reporting.* ***monthlyIncome*** *= Stores the total money earned by the user.* ***fixedexpenses*** *= Stores non-changing costs (e.g., rent).* ***variableAmount*** *= Stores individual costs that change monthly.* ***totalExpenses*** *= An accumulator that stores the sum of all costs.* ***isEmergency*** *= Stores a True/False value to flag urgent spending.* ***remainingBudget*** *= The final result of (Income - Expenses).*

References

Chaudhuri, A. B. (2020). *Flowchart and algorithm basics: The art of programming*. Mercury Learning & Information.

Computer Science Tutor. (2020, June 12). *Errors in programs part 1 - Types of errors & debugging tips* [Video]. YouTube. https://youtu.be/4kYtCVneais

Computer Science Tutor. (2022, January 5). *Control structures - Sequence, selection & iteration explained* [Video]. YouTube. https://youtu.be/Yw2Rxa6gfVg

MV, T. (2019, November 12). *What exactly is a programming paradigm?* freeCodeCamp. https://www.freecodecamp.org/news/what-exactly-is-a-programming-paradigm/

Ndjountche, T. (2016). *Digital electronics 1: Combinational logic circuits*. John Wiley & Sons.

Simplilearn. (2021, September 7). *Problem solving in programming | Problem solving skills for programming | Simplilearn* [Video]. YouTube. https://youtu.be/-c5XAmA5Oc4